

One base per object:
a machine-checked zero-knowledge proof of possession
for every mathematical object of the development

Generated from `RequestProject/ZKPoP/` and `tables/zkpop_bases.tsv`

Protocol ZKPoP/1 on a cyclic base of arbitrary size

Abstract

This report accompanies the Lean 4 modules `RequestProject/ZKPoP/`, which apply a zero-knowledge proof of possession (ZKPoP) to the mathematical content of this development: each of the 88 objects the project carries — a sporadic group order, a maximal-subgroup order, a lattice kissing number, a code size, a design’s block count, a moonshine coefficient — is turned into the *base* of a proof of possession of its own. The Schnorr Σ -protocol for possession of a discrete logarithm is formalised over a cyclic base of *arbitrary* size and proved, unconditionally and with no `sorry`, to be complete, specially sound with an explicit extractor, and perfectly honest-verifier zero knowledge; its knowledge error is computed exactly. The arithmetic of the base is then settled: on an even base at least half of all challenges are unusable by the extractor, so the base attached to an object is the largest prime factor of its size rather than the size itself, and for a group object that prime base is realised inside the object by Cauchy’s theorem. No computational-hardness claim is made anywhere: which bases make the discrete logarithm *hard* is a separate question this development does not address.

1 What is claimed

The development supports exactly the following composition.

1. **A protocol on a base of any size** (`ZKPoP/Protocol.lean`). Fix a finite cyclic group $G = \langle g \rangle$ of order $n > 1$ in which every element is killed by n (`GoodBase`). Exponents are elements of $\mathbb{Z}/n$ acting through $\text{gp}(y, a) = y^{a.\text{val}}$. For the statement h and claimed witness x with $h = g^x$:

$$\text{commitment } a = g^r, \quad \text{challenge } c, \quad \text{response } s = r + cx, \quad \text{check } g^s = a \cdot h^c.$$

Completeness, special soundness and perfect honest-verifier zero knowledge are proved unconditionally, and the knowledge error of k rounds on a prime base is computed to be exactly n^{-k} .

2. **Which numbers are good bases** (`ZKPoP/Bases.lean`). Soundness is only as good as the supply of *invertible* challenge differences in $\mathbb{Z}/n$. On an even base at least half of all challenges are unusable, so an even base has knowledge error at least 1/2 however astronomically large it is. Every group order in this project is even; hence the reduction of an object to the largest prime factor of its size, which the project’s existing factorization certificates justify.

3. **The objects** (ZKPoP/Objects.lean). A corpus of 88 objects of the development, each with its size, a certified prime factorization of that size, and its base. Every row is checked; every object then carries a complete proof of possession on its own base, with a recorded number of rounds bringing its knowledge error below 2^{-80} .

What is *not* claimed: that any of this makes a discrete logarithm hard. Completeness, soundness and zero knowledge here are unconditional combinatorial statements about the protocol. A base of size 2 (the binary Golay code) is as complete, as specially sound and as zero-knowledge as a base of size 1823 (the moonshine coefficient 196884); what separates them is only the knowledge error per round, $1/2$ against $1/1823$, and hence the number of repetitions in the last column of the object table.

2 The protocol, precisely

Base	$G = \langle g \rangle$ commutative, $\text{ord}(g) = n > 1$, $y^n = 1$ for all $y \in G$
Exponent action	$\text{gp}(y, a) = y^{a.\text{val}}$ for $a \in \mathbb{Z}/n$
Statement	$h \in G$; the prover claims to know x with $h = \text{gp}(g, x)$
Commitment	$a = \text{gp}(g, r)$, $r \in \mathbb{Z}/n$ uniform
Challenge	$c \in \mathbb{Z}/n$
Response	$s = r + cx$
Verification	$\text{gp}(g, s) = a \cdot \text{gp}(h, c)$
Extractor	$x = (c_1 - c_2)^{-1}(s_1 - s_2)$ from two accepting transcripts sharing a
Simulator	$(\text{gp}(g, s) \cdot \text{gp}(h, c)^{-1}, c, s)$, which never sees x
Knowledge error	n^{-k} after k rounds on a prime base

That gp is a genuine $\mathbb{Z}/n$ -action on the whole base group — and not only on powers of g — is what lets special soundness be stated with *no* hypothesis on the claimed statement h : the verifier does not know where h came from, and does not need to.

3 Machine-checked results

All 22 entries below are proved in the accompanying Lean modules, with no `sorry`.

Declaration	Module	Statement
<code>goodBase_zmod</code>	Protocol	A base of every size $n > 1$ exists: $\mathbb{Z}/n$ written multiplicatively, generated by 1. Nothing below is vacuous.
<code>accepts_honest</code>	Protocol	Completeness. A prover holding x with $h = g^x$ is accepted on every randomness r and every challenge c , on a base of any size.
<code>special_soundness</code>	Protocol	Special soundness. Two accepting transcripts with the same commitment whose challenges differ by a unit of $\mathbb{Z}/n$ determine the witness: the explicit extractor $(c_1 - c_2)^{-1}(s_1 - s_2)$ returns a discrete logarithm of the statement. No hypothesis is placed on the statement h .
<code>simulate_accepts</code>	Protocol	The witness-free simulator produces accepting transcripts for <i>every</i> statement.

Declaration	Module	Statement
<code>simulate_honest</code>	Protocol	Perfect simulation. Run at response $r + cx$ the simulator reproduces exactly the honest transcript with randomness r .
<code>hvzk_ncard</code>	Protocol	Perfect honest-verifier zero knowledge , in counting form: for each challenge, every transcript has as many honest random tapes as simulator tapes, so the two distributions agree fibre by fibre.
<code>hvzk_bijective</code>	Protocol	The reindexing behind it: $r \mapsto r + cx$ is a bijection of the randomness space carrying honest transcripts onto simulated ones.
<code>card_accepting_le_one</code>	Protocol	Knowledge error of one round. On a base of prime size n , a prover with no witness (h not a power of g) is accepted on at most one of the n challenges, whatever commitment it sends and whatever responses it prepares.
<code>card_accepting_pow_le</code>	Protocol	The same over k independent rounds: at most one of the n^k challenge vectors is answered, so the knowledge error after k rounds is n^{-k} .
<code>card_nonUnit_diff_ge_half</code>	Bases	Even bases are bad. If $2 \mid n$ then, against any fixed challenge, at least $n/2$ challenges give a non-invertible difference, so the extractor cannot be run: one round on an even base has knowledge error at least $1/2$, however large n is.
<code>topPrime_isGreatest</code>	Bases	For a certified factorization, the largest prime it records is a prime divisor of the number and dominates every prime divisor of it.
<code>knowledgeError_le_of_le</code>	Bases	$\text{knowledgeError}(b, k) = b^{-k}$ is antitone in both arguments: a bigger base and more rounds can only help.
<code>objects_rowValid</code>	Objects	Every one of the 88 rows is arithmetically honest: the recorded factorization is a genuine prime factorization of the recorded size, and the recorded base is the largest prime in it.
<code>base_prime, base_dvd_size,</code> <code>base_greatest</code>	Objects	Consequently each object's base is a prime divisor of its size and dominates every prime divisor of its size.
<code>sporadic_sizes_even</code>	Objects	Every sporadic group order in the corpus is even — which, with <code>card_nonUnit_diff_ge_half</code> , is exactly why the base is the largest prime factor of the size and not the size itself.
<code>base_zkpop</code>	Objects	Each object carries a full proof of possession on its own base: completeness, special soundness with the extractor usable on <i>every</i> pair of distinct challenges (the base is prime), perfect honest-verifier zero knowledge, and knowledge error $1/\text{base}$ per round.
<code>exists_goodBase_in_group</code>	Objects	The base is structural, not just arithmetic. Any finite group whose order is a group object's recorded size contains, by Cauchy's theorem, an element of order exactly that object's base, and the cyclic subgroup it generates is a legitimate base for the protocol.
<code>bases_range, bases_card,</code> <code>bases_sorted_distinct</code>	Objects	The census of bases: 19 distinct values, from 2 to 1823.
<code>supersingular_primes_are_bases</code>	Objects	All fifteen primes dividing the order of the Monster — the supersingular primes — occur as bases of objects of the corpus.

Declaration	Module	Statement
<code>objectKnowledgeError_mono</code>	Objects	Comparing two objects at equal round count, the one with the larger base has the smaller knowledge error.
<code>rounds80_pow</code> , <code>rounds80_minimal</code>	Objects	<code>rounds80(b)</code> is the <i>least</i> k with $2^{80} \leq b^k$: the recorded round count both suffices and is minimal.
<code>objectKnowledgeError_rounds80</code>	Objects	The security budget of every object. Repeating an object’s proof rounds80 times brings its knowledge error below 2^{-80} .

4 The corpus

The 88 objects are drawn from the rest of the development and span 19 distinct bases, from 2 to 1823.

Domain	Objects	Base	Objects	Error/round	Rounds
Monster maximal subgroup	46	2	2	1/2	80
sporadic group order	26	3	2	1/3	51
lattice	3	5	5	1/5	35
crystals	3	7	9	1/7	29
code	2	11	11	1/11	24
prime fiber	2	13	13	1/13	22
group order	1	17	4	1/17	20
design	1	19	7	1/19	19
representation	1	23	12	1/23	18
moonshine	1	29	4	1/29	17
character table	1	31	6	1/31	17
subgroup lattice	1	41	3	1/41	15
		43	1	1/43	15
		47	2	1/47	15
		59	1	1/59	14
		67	1	1/67	14
		71	3	1/71	14
		97	1	1/97	13
		1823	1	1/1823	8

“Rounds” is the least k with $2^{80} \leq \text{base}^k$, i.e. the number of repetitions that brings the knowledge error below 2^{-80} ; that it suffices and that one round fewer does not are both checked in Lean.

5 Object table

Object	Size	Factorization	Base	Rounds
M11	7920	$2^4 \cdot 3^2 \cdot 5 \cdot 11$	11	24
M12	95040	$2^6 \cdot 3^3 \cdot 5 \cdot 11$	11	24
M22	443520	$2^7 \cdot 3^2 \cdot 5 \cdot 7 \cdot 11$	11	24
M23	10200960	$2^7 \cdot 3^2 \cdot 5 \cdot 7 \cdot 11 \cdot 23$	23	18

Object	Size	Factorization	Base	Rounds
M24	244823040	$2^{10} \cdot 3^3 \cdot 5 \cdot 7 \cdot 11 \cdot 23$	23	18
J1	175560	$2^3 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 19$	19	19
J2	604800	$2^7 \cdot 3^3 \cdot 5^2 \cdot 7$	7	29
J3	50232960	$2^7 \cdot 3^5 \cdot 5 \cdot 17 \cdot 19$	19	19
J4	86 775 571 046 077 562 880	$2^{21} \cdot 3^3 \cdot 5 \cdot 7 \cdot 11^3 \cdot 23 \cdot 29 \cdot 31 \cdot 37 \cdot 43$	43	15
Co1	4 157 776 806 543 360 000	$2^{21} \cdot 3^9 \cdot 5^4 \cdot 7^2 \cdot 11 \cdot 13 \cdot 23$	23	18
Co2	42 305 421 312 000	$2^{18} \cdot 3^6 \cdot 5^3 \cdot 7 \cdot 11 \cdot 23$	23	18
Co3	495766656000	$2^{10} \cdot 3^7 \cdot 5^3 \cdot 7 \cdot 11 \cdot 23$	23	18
Fi22	64 561 751 654 400	$2^{17} \cdot 3^9 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13$	13	22
Fi23	4 089 470 473 293 004 800	$2^{18} \cdot 3^{13} \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 23$	23	18
Fi24'	1 255 205 709 190 661 721 292 800	$2^{21} \cdot 3^{16} \cdot 5^2 \cdot 7^3 \cdot 11 \cdot 13 \cdot 17 \cdot 23 \cdot 29$	29	17
HS	44352000	$2^9 \cdot 3^2 \cdot 5^3 \cdot 7 \cdot 11$	11	24
McL	898128000	$2^7 \cdot 3^6 \cdot 5^3 \cdot 7 \cdot 11$	11	24
He	4030387200	$2^{10} \cdot 3^3 \cdot 5^2 \cdot 7^3 \cdot 17$	17	20
Ru	145926144000	$2^{14} \cdot 3^3 \cdot 5^3 \cdot 7 \cdot 13 \cdot 29$	29	17
Suz	448345497600	$2^{13} \cdot 3^7 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13$	13	22
O'N	460815505920	$2^9 \cdot 3^4 \cdot 5 \cdot 7^3 \cdot 11 \cdot 19 \cdot 31$	31	17
HN	273 030 912 000 000	$2^{14} \cdot 3^6 \cdot 5^6 \cdot 7 \cdot 11 \cdot 19$	19	19
Ly	51 765 179 004 000 000	$2^8 \cdot 3^7 \cdot 5^6 \cdot 7 \cdot 11 \cdot 31 \cdot 37 \cdot 67$	67	14
Th	90 745 943 887 872 000	$2^{15} \cdot 3^{10} \cdot 5^3 \cdot 7^2 \cdot 13 \cdot 19 \cdot 31$	31	17
B	4 154 781 481 226 426 191 177 580 544 000 000	$2^{41} \cdot 3^{13} \cdot 5^6 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 31 \cdot 47$	47	15
M	808 017 424 794 512 875 886 459 904 961 710 757 005 754 368 000 000 000	$2^{46} \cdot 3^{20} \cdot 5^9 \cdot 7^6 \cdot 11^2 \cdot 13^3 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 41 \cdot 47 \cdot 59 \cdot 71$	71	14
2.B	8 309 562 962 452 852 382 355 161 088 000 000	$2^{42} \cdot 3^{13} \cdot 5^6 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 31 \cdot 47$	47	15
2^(1+24).Co1	139 511 839 126 336 328 171 520 000	$2^{46} \cdot 3^9 \cdot 5^4 \cdot 7^2 \cdot 11 \cdot 13 \cdot 23$	23	18
3.Fi24'	7 531 234 255 143 970 327 756 800	$2^{22} \cdot 3^{17} \cdot 5^2 \cdot 7^3 \cdot 11 \cdot 13 \cdot 17 \cdot 23 \cdot 29$	29	17
2^(2.2)E6(2):S3	1 836 779 512 410 596 494 540 800	$2^{39} \cdot 3^{10} \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19$	19	19
2^(10+16).010+(2)	1 577 011 055 923 770 163 200	$2^{46} \cdot 3^5 \cdot 5^2 \cdot 7 \cdot 17 \cdot 31$	31	17
2^(2+11+22).(S3 x M24)	50 472 333 605 150 392 320	$2^{46} \cdot 3^4 \cdot 5 \cdot 7 \cdot 11 \cdot 23$	23	18
3^(1+12).2.Suz.2	2 859 230 155 080 499 200	$2^{15} \cdot 3^{20} \cdot 5^2 \cdot 7 \cdot 11 \cdot 13$	13	22
2^(5+10+20).(S3 x L5(2))	2 061 452 360 684 666 880	$2^{46} \cdot 3^3 \cdot 5 \cdot 7 \cdot 31$	31	17
S3 x Th	544 475 663 327 232 000	$2^{16} \cdot 3^{11} \cdot 5^3 \cdot 7^2 \cdot 13 \cdot 19 \cdot 31$	31	17
2^(3+6+12+18).(L3(2) x 3.S6)	199 495 389 743 677 440	$2^{46} \cdot 3^4 \cdot 5 \cdot 7$	7	29

Object	Size	Factorization	Base	Rounds
3 ⁸ .08-(3).2_3	133 214 132 225 341	2 ¹ · 3 ²⁰ · 5 · 7 · 13 · 41	41	15
	440			
(D10 x HN).2	5 460 618 240 000 000	2 ¹ · 3 ⁶ · 5 ⁷ · 7 · 11 · 19	19	19
(3 ² :2 x 08+(3)).S4	2 139 341 679 820 800	2 ¹ · 3 ¹⁵ · 5 ² · 7 · 13	13	22
3 ² (2+5+10).(M11 x 2.S4)	49 093 924 366 080	2 ⁸ · 3 ²⁰ · 5 · 11	11	24
3 ² (3+2+6+6):(L3(3) x SD16)	11 604 018 486 528	2 ⁸ · 3 ²⁰ · 13	13	22
5 ² (1+6):2.J2:4	378000000000	2 ¹⁰ · 3 ³ · 5 ⁹ · 7	7	29
(7:3 x He):2	169276262400	2 ¹¹ · 3 ⁴ · 5 ² · 7 ⁴ · 17	17	20
(A5 x A12):2	28740096000	2 ¹² · 3 ⁶ · 5 ³ · 7 · 11	11	24
5 ² (3+3).(2 x L3(5))	11625000000	2 ⁶ · 3 · 5 ⁹ · 31	31	17
(A6 x A6 x A6).(2 x S4)	2239488000	2 ¹³ · 3 ⁷ · 5 ³	5	35
(A5 x U3(8):3_1):2	1985679360	2 ¹² · 3 ⁶ · 5 · 7 · 19	19	19
5 ² (2+2+4):(S3 x GL2(5))	1125000000	2 ⁶ · 3 ² · 5 ⁹	5	35
(L3(2) x S4(4):2).2	658022400	2 ¹³ · 3 ³ · 5 ² · 7 · 17	17	20
7 ² (1+4):(3 x 2.S7)	508243680	2 ⁵ · 3 ³ · 5 · 7 ⁶	7	29
(5 ² :4.2 ² x U3(5)).S3	302400000	2 ⁹ · 3 ³ · 5 ⁵ · 7	7	29
(L2(11) x M12):2	125452800	2 ⁹ · 3 ⁴ · 5 ² · 11 ²	11	24
(A7 x (A5 x A5):2 ²):2	72576000	2 ¹⁰ · 3 ⁴ · 5 ³ · 7	7	29
5 ⁴ :(3 x 2.L2(25)):2_2	58500000	2 ⁵ · 3 ² · 5 ⁶ · 13	13	22
7 ² (2+1+2):GL2(7)	33882912	2 ⁵ · 3 ² · 7 ⁶	7	29
M11 x A6.2 ²	11404800	2 ⁹ · 3 ⁴ · 5 ² · 11	11	24
(S5 x S5 x S5):S3	10368000	2 ¹⁰ · 3 ⁴ · 5 ³	5	35
(L2(11) x L2(11)):4	1742400	2 ⁶ · 3 ² · 5 ² · 11 ²	11	24
13 ² :2.L2(13).4	1476384	2 ⁵ · 3 · 7 · 13 ³	13	22
(7 ² :2:(3 x 2.A4) x L2(7)):2	1185408	2 ⁷ · 3 ³ · 7 ³	7	29
(13:6 x L3(3)).2	876096	2 ⁶ · 3 ⁴ · 13 ²	13	22
13 ² (1+2):(3 x 4.S4)	632736	2 ⁵ · 3 ² · 13 ³	13	22
U3(4):4	249600	2 ⁸ · 3 · 5 ² · 13	13	22
L2(71)	178920	2 ³ · 3 ² · 5 · 7 · 71	71	14
11 ² :2:(5 x 2.A5)	72600	2 ³ · 3 · 5 ² · 11 ²	11	24
L2(41)	34440	2 ³ · 3 · 5 · 7 · 41	41	15
L2(29):2	24360	2 ³ · 3 · 5 · 7 · 29	29	17
7 ² :SL2(7)	16464	2 ⁴ · 3 · 7 ³	7	29
L2(19):2	6840	2 ³ · 3 ² · 5 · 19	19	19
L2(13):2	2184	2 ³ · 3 · 7 · 13	13	22
59:29	1711	29 · 59	59	14
41:40	1640	2 ³ · 5 · 41	41	15
Co0 = 2.Co1	8 315 553 613 086 720	2 ² · 3 ⁹ · 5 ⁴ · 7 ² · 11 · 13 · 23	23	18
	000			
Leech kissing number	196560	2 ⁴ · 3 ³ · 5 · 7 · 13	13	22
Leech lattice rank	24	2 ³ · 3	3	51
E8 root count	240	2 ⁴ · 3 · 5	5	35
binary Golay code size	4096	2 ¹²	2	80
extended ternary Golay code size	729	3 ⁶	3	51
S(5,8,24) block count	759	3 · 11 · 23	23	18
Monster minimal faithful degree	196883	47 · 59 · 71	71	14
j-coefficient c(1)	196884	2 ² · 3 ³ · 1823	1823	8
Monster conjugacy classes	194	2 · 97	97	13
Monster maximal-subgroup classes	46	2 · 23	23	18

Object	Size	Factorization	Base	Rounds
supersingular prime count	15	$3 \cdot 5$	5	35
six-prime primorial	30030	$2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13$	13	22
crystallographic point groups	32	2^5	2	80
space groups	230	$2 \cdot 5 \cdot 23$	23	18
wallpaper groups	17	17	17	20

6 Axiom closure

Captured from lake `env lean scripts/print_zkpop_axioms.lean`. Alongside Lean’s standard axioms, the finite checks of the object table (the factorizations, the census of bases, the round counts) are discharged by compiled evaluation and so carry `Lean.ofReduceBool` and `Lean.trustCompiler`, as elsewhere in this development.

```
'RequestProject.ZKPoP.goodBase_zmod' depends on axioms: [propext, Classical.choice,
  Quot.sound]
'RequestProject.ZKPoP.accepts_honest' depends on axioms: [propext, Classical.choice,
  Quot.sound]
'RequestProject.ZKPoP.simulate_accepts' depends on axioms: [propext]
'RequestProject.ZKPoP.simulate_honest' depends on axioms: [propext, Classical.choice,
  Quot.sound]
'RequestProject.ZKPoP.hvzk_ncard' depends on axioms: [propext, Classical.choice, Quot.sound]
'RequestProject.ZKPoP.hvzk_bijective' depends on axioms: [propext, Classical.choice,
  Quot.sound]
'RequestProject.ZKPoP.special_soundness' depends on axioms: [propext, Classical.choice,
  Quot.sound]
'RequestProject.ZKPoP.isUnit_sub_of_prime' depends on axioms: [propext, Classical.choice,
  Quot.sound]
'RequestProject.ZKPoP.accepting_subsingleton' depends on axioms: [propext, Classical.choice,
  Quot.sound]
'RequestProject.ZKPoP.card_accepting_le_one' depends on axioms: [propext, Classical.choice,
  Quot.sound]
'RequestProject.ZKPoP.card_accepting_pow_le' depends on axioms: [propext, Classical.choice,
  Quot.sound]
'RequestProject.ZKPoP.card_nonUnit_diff_ge_half' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'RequestProject.ZKPoP.topPrime_isGreatest' depends on axioms: [propext, Classical.choice,
  Quot.sound]
'RequestProject.ZKPoP.knowledgeError_pos' depends on axioms: [propext, Classical.choice,
  Quot.sound]
'RequestProject.ZKPoP.knowledgeError_le_of_le' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'RequestProject.ZKPoP.objects_length' depends on axioms: [Lean.ofReduceBool,
  Lean.trustCompiler]
'RequestProject.ZKPoP.objects_rowValid' depends on axioms: [propext, Classical.choice,
  Lean.ofReduceBool, Lean.trustCompiler, Quot.sound]
'RequestProject.ZKPoP.base_prime' depends on axioms: [propext, Classical.choice,
  Lean.ofReduceBool, Lean.trustCompiler, Quot.sound]
'RequestProject.ZKPoP.base_dvd_size' depends on axioms: [propext, Classical.choice,
  Lean.ofReduceBool, Lean.trustCompiler, Quot.sound]
'RequestProject.ZKPoP.base_greatest' depends on axioms: [propext, Classical.choice,
  Lean.ofReduceBool, Lean.trustCompiler, Quot.sound]
'RequestProject.ZKPoP.bases_range' depends on axioms: [Lean.ofReduceBool,
  Lean.trustCompiler]
```

```

'RequestProject.ZKPoP.bases_card' depends on axioms: [Lean.ofReduceBool, Lean.trustCompiler]
'RequestProject.ZKPoP.bases_sorted_distinct' depends on axioms: [propext, Lean.ofReduceBool,
    Lean.trustCompiler, Quot.sound]
'RequestProject.ZKPoP.supersingular_primes_are_bases' depends on axioms: [propext,
    Lean.ofReduceBool, Lean.trustCompiler, Quot.sound]
'RequestProject.ZKPoP.sporadic_sizes_even' depends on axioms: [propext, Lean.ofReduceBool,
    Lean.trustCompiler]
'RequestProject.ZKPoP.base_zkpop' depends on axioms: [propext, Classical.choice,
    Lean.ofReduceBool, Lean.trustCompiler, Quot.sound]
'RequestProject.ZKPoP.objectKnowledgeError_mono' depends on axioms: [propext,
    Classical.choice, Lean.ofReduceBool, Lean.trustCompiler, Quot.sound]
'RequestProject.ZKPoP.rounds80_pow' depends on axioms: [propext, Lean.ofReduceBool,
    Lean.trustCompiler]
'RequestProject.ZKPoP.rounds80_minimal' depends on axioms: [propext, Lean.ofReduceBool,
    Lean.trustCompiler]
'RequestProject.ZKPoP.objectKnowledgeError_rounds80' depends on axioms: [propext,
    Classical.choice, Lean.ofReduceBool, Lean.trustCompiler, Quot.sound]
'RequestProject.ZKPoP.exists_goodBase_in_group' depends on axioms: [propext,
    Classical.choice, Lean.ofReduceBool, Lean.trustCompiler, Quot.sound]

```

7 Trust boundary

The following are outside what is proved here.

- **Hardness.** Nothing here says that computing a discrete logarithm on any of these bases is hard — on these bases it is not: they are tiny. The protocol’s zero knowledge is perfect and unconditional, and its soundness is an extraction statement, but a proof of possession is only *convincing* in a group where the discrete logarithm is infeasible. This report claims a machine-checked protocol and an exact knowledge error, not a deployable cryptosystem.
- **Repetition model.** `card_accepting_pow_1e` counts accepting challenge vectors for a prover that fixes its commitments and responses in advance; the probabilistic reading n^{-k} presumes challenges drawn uniformly and independently. No probability space is formalised: the results are the exact counting statements, and the probabilities are their immediate reading.
- **Fiat–Shamir.** The protocol is interactive here. Nothing about the random-oracle heuristic, hashing, or a non-interactive variant is modelled.
- **The corpus data.** Sizes are the mathematical invariants recorded elsewhere in this development; the paper’s arithmetic claims about them (factorization, largest prime, round count) are all machine-checked, but the attribution of a size to a named object rests on the same sources as the rest of the project.
- **Compiled evaluation.** The finite checks over the 88 rows use `native_decide`, hence trust the Lean compiler, as recorded in the axiom closure above.

8 Reproducing

```

lake build RequestProject.ZKPoP.Objects          # re-check every theorem of the development
python3 scripts/generate_zkpop_bases_table.py    # regenerate tables/zkpop_bases.tsv
python3 scripts/capture_zkpop_axioms.py          # regenerate paper/axioms.txt
python3 scripts/make_zkpop_paper.py              # regenerate paper/zkpop.tex and this PDF

```